

Architecture of the Current GBT+JEPA Market Direction System and 0DTE Option Execution Stack

Kripta Studios

June 2026

Contents

1	Research Question	2
1.1	Hypothesis	2
1.2	Non-Hypothesis	2
2	High-Level System Diagram	3
3	Pipeline Stages	3
4	Data Model	4
5	XInputJEPA Architecture	4
5.1	Motivation	4
5.2	Encoders	5
5.3	Multi-Horizon Latent Predictor	5
5.4	Auxiliary Direction Head	5
5.5	Training Objective	5
6	JEPA Feature Export	6
7	GBT Direction Model	6
7.1	Why LightGBM Remains the Final Predictor	6
7.2	Feature Modes	7
7.3	Classifier Specification	7
8	Thresholding and Trading Contract	7
9	0DTE Option Execution Layer	8
9.1	Fixed-Delta Production Rule	8
9.2	Why Fixed Delta Can Beat Learned Selection	8
10	OptionValueJEPA Research Architecture	8
11	Oracle Definitions	9

12 Live Runtime	9
13 Empirical Summary	9
14 Risk Controls and Bias Controls	10
15 Conclusion	10

Abstract

This document describes the current production-candidate architecture after the research migration from the legacy GBT+PPO/RL stack to a supervised GBT+JEPA stack. The deployed idea is not “JEPA instead of everything.” The promoted model is a hybrid: a tabular LightGBM classifier predicts 180 minute market direction using both engineered market microstructure features and latent features learned by an explicit-input Joint-Embedding Predictive Architecture (XInputJEPA). The option layer currently uses a deterministic 0DTE fixed-delta execution rule near $\Delta = 0.70$. OptionValueJEPA, learned strike selection, and learned 5 minute exits were evaluated as research extensions, but they are not yet promoted because they did not beat the fixed-delta baseline on the relevant out-of-sample gates. The strict out-of-sample result is the April–May 2026 frozen-March test, where `base_jepa` produced 105 trades, 68.6% win rate, 1.945 profit factor, and \$9,132 proxy PnL at 1 bps cost.

1 Research Question

The architectural question is whether a JEPA-style latent state adds predictive information to an already feature-rich GBT market model. The specific trading question is:

$$\Pr(S_{t+180m} > S_t \mid \mathcal{F}_t),$$

where S_t is the underlying spot or proxy price and $\mathcal{F}_t$ is the information set available at timestamp t . The system trades a fixed 180 minute horizon. This horizon is not arbitrary: it matches the label produced by the data collector and the operational window used by the existing trading pipeline.

1.1 Hypothesis

The hypothesis is that option-market state is partially observable. Hand-engineered features describe the current surface and exposure map, but they may not fully encode how the system is evolving. JEPA is used to learn a compact latent transition model:

$$\text{current state} + \text{recent input} \rightarrow \text{future latent state}.$$

If the latent variables summarize transition dynamics that the GBT cannot easily reconstruct from static features, then adding JEPA features should improve out-of-sample trade metrics. This is exactly what the strict April–May 2026 test shows for the combined `base_jepa` model.

1.2 Non-Hypothesis

The research does not currently support replacing the entire trading system with an end-to-end JEPA-only model. The JEPA-only model has signal, but it underperforms the combined model. The production design therefore keeps the interpretable engineered state and uses JEPA as a representation layer.

2 High-Level System Diagram

ThetaData and market data → base feature parquet → XInputJEPA append
 GBT feature matrix → per-ticker 180m direct

The code-level orchestration is:

```
neural/jepa/run_pipeline.ps1
```

The production mode is:

```
.\neural\jepa\run_pipeline.ps1 -ProductionTrain
```

When `-SkipCollect` is not supplied, this mode runs the data collector. The collector itself decides whether to rebuild or append incrementally.

3 Pipeline Stages

Stage	Main script	Function
Data collection	neural/collect_training_data_spx_qqq.py	Builds the base SPX/QQQ/SPY market-state parquet. It can rebuild from 2022 or append only new dates when the collector script is older than the output parquet.
Production cutoff	neural/jepa/run_pipeline.ps1	In production mode, creates a cutoff parquet using <code>TrainEndDate=20261230</code> ; the real maximum date is the maximum date available in the source data.
XInputJEPA training	neural/jepa/train_xinput_jepa.py	Trains a state/input JEPA using 24 five-minute contexts and multi-horizon latent prediction. In production mode it selects a validation epoch and then refits on all available dates.
Feature append	neural/jepa/append_xinput_jepa_features.py	Appends 42 xjepa_* features to each live-safe row.
180m GBT training	neural/jepa/train_180m_production.py	Trains per-ticker LightGBM classifiers for base, jepa_only, and base_jepa.
180m backtesting	neural/jepa/backtest_jepa_180m.py	Scores a frozen model over a chosen date range and simulates fixed 180 minute holds with cooldown.
Option policy research	neural/jepa/train_backtest_option_policy.py	Builds 0DTE option candidate labels and evaluates fixed delta, learned selectors, and oracle ceilings.

Stage	Main script	Function
OptionValue-JEPA research	neural/jepa/train_option_value_jepa.py and neural/jepa/walkforward_option_value_jepa.py	Learns candidate option values and 5 minute continuation values. This is research-only until it beats fixed delta in walk-forward.

4 Data Model

The base data object is a row-level intraday market state. Required identifiers include ticker, date, time, and spot price. The training rows are sorted by ticker, date, and timestamp or time. The exact 180 minute label is built by shifting within each ticker-date group:

$$S_{t+180m} = S_{i+36},$$

where one row step is five minutes. Rows that do not have an exact 36-step future observation are excluded from 180 minute training.

The latest production run used:

- 221,832 rows in the production source parquet,
- source maximum date May 29, 2026,
- 56,160 rows after exact 180 minute label construction,
- 960 production dates and 56,160 JEPA windows in the final-fit XInputJEPA run.

5 XInputJEPA Architecture

5.1 Motivation

The most important architectural change versus a conventional tabular model is the state/input split. Market state is not treated as a single flat vector. Instead, the model receives two synchronized sequences:

$$X_t = [x_{t-23}, \dots, x_t], \quad U_t = [u_{t-23}, \dots, u_t],$$

where x is the state feature set and u is the exogenous-input/change feature set. In production:

- x has 155 dimensions,
- u has 25 dimensions,
- the context length is 24 rows, or approximately two hours at five-minute spacing.

5.2 Encoders

Both the state and input branches use the same sequence-encoder pattern:

$$\text{LayerNorm} \rightarrow \text{GRU}_2 \text{ layers} \rightarrow \text{LayerNorm} \rightarrow \text{Linear} \rightarrow \text{GELU} \rightarrow \text{Dropout} \rightarrow \text{Linear}.$$

The state encoder produces a 16-dimensional latent vector:

$$z_t = f_\theta(X_t) \in \mathbb{R}^{16}.$$

The input encoder produces a 12-dimensional latent vector:

$$u_t = g_\phi(U_t) \in \mathbb{R}^{12}.$$

5.3 Multi-Horizon Latent Predictor

The predictor learns a separate head for each horizon. For horizon index h , an embedding e_h is concatenated with the state and input latents:

$$\hat{z}_{t+h} = p_{\psi,h}([z_t, u_t, e_h]).$$

The production horizons are:

$$h \in \{1, 3, 6, 12, 24, 36\}.$$

At five-minute spacing, the final horizon corresponds to 180 minutes.

5.4 Auxiliary Direction Head

The model also includes a three-class head over short, hold, and long:

$$\ell_t = c_\omega([z_t, u_t, \text{mean}_h(\hat{z}_{t+h})]) \in \mathbb{R}^3.$$

This head does not replace the GBT. It regularizes the latent space toward trading-relevant state transitions and produces useful appended features such as `xjepa_prob_short`, `xjepa_prob_hold`, and `xjepa_prob_long`.

5.5 Training Objective

The loss is a weighted combination:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{sig}} \mathcal{L}_{\text{sigreg}} + \lambda_{\text{vic}} \mathcal{L}_{\text{vicreg}} + \lambda_{\text{ce}} \mathcal{L}_{\text{ce}}.$$

The predictive loss aligns predicted future latent states with encoded future contexts:

$$\mathcal{L}_{\text{pred}} = \|\hat{z}_{t+h} - \text{stopgrad}(f_\theta(X_{t+h}))\|_2^2.$$

The SIGReg and VICReg-style terms prevent latent collapse. The cross-entropy term is weighted by class frequency and uses a balanced sampler when requested by the pipeline. The production hyperparameters are:

Parameter	Value
Hidden dimension	96
GRU layers	2
Dropout	0.15
z dimension	16
u dimension	12
Learning rate	0.0007
Weight decay	0.02
Batch size	512
λ_{sig}	0.15
λ_{vic}	0.20
λ_{ce}	0.35

Table 2: Production XInputJEPA hyperparameters from the production configuration artifact.

6 JEPA Feature Export

After training, `append_xinput_jepa_features.py` appends 42 live-safe features. These include latent components, latent velocities, prediction dispersion, lagged prediction errors, class probabilities, entropy, confidence, and context validity.

The lagged prediction error features are important because they measure whether the previous JEPA forecast matched the state that actually arrived:

$$e_{t,h} = \|\hat{z}_{t-h,h} - z_t\|_2.$$

This is still live-safe at time t because it uses a prediction made at $t - h$ and the current observed latent state.

7 GBT Direction Model

7.1 Why LightGBM Remains the Final Predictor

The GBT has three advantages in this project:

1. It handles heterogeneous tabular features well.
2. It can ignore weak or redundant features.
3. It gives stable per-ticker thresholding with modest data volume.

The JEPA block produces a learned representation, but the final decision boundary is still learned by the GBT over both explicit features and latents.

7.2 Feature Modes

Three feature modes are trained:

- base: original engineered market features only.
- jepa_only: JEPA features plus basic time context.
- base_jepa: base market features plus JEPA features.

The live model is base_jepa. The production artifact has 222 columns: 180 base columns present in the production training table plus 42 JEPA columns.

7.3 Classifier Specification

The 180 minute classifier is a per-ticker LightGBM binary classifier:

$$\hat{p}_t = \Pr(y_t = 1 \mid \mathbf{x}_t).$$

The implementation uses:

- binary objective,
- 0.035 learning rate,
- 31 leaves,
- 0.85 row subsampling,
- 0.85 column subsampling,
- minimum child samples of 40,
- L1 regularization 0.05,
- L2 regularization 0.50,
- balanced class weights.

Missing values are imputed with training medians stored in the joblib artifact. This matters for live inference because the bot can reproduce the exact training matrix behavior.

8 Thresholding and Trading Contract

The model does not trade merely because $\hat{p}_t > 0.5$. Each ticker receives thresholds selected on validation data:

$$\text{LONG} \quad \text{if} \quad \hat{p}_t \geq \tau_L,$$

$$\text{SHORT} \quad \text{if} \quad \hat{p}_t \leq \tau_S,$$

where $\tau_S = 1 - \tau_L$ in the threshold grid. Candidate long thresholds include 0.52, 0.55, 0.57, 0.60, 0.62, 0.65, and 0.70. The validation score is:

$$\text{score} = \text{PnL} - 0.25|\text{MaxDD}|,$$

subject to a minimum validation trade count.

The promoted contract is a fixed 180 minute hold with a 180 minute cooldown. The cooldown prevents overlapping entries that are effectively the same forecast window.

9 0DTE Option Execution Layer

9.1 Fixed-Delta Production Rule

The promoted 0DTE execution rule is deterministic:

1. use `base_jepa` to choose direction,
2. buy the 0DTE call for a long signal or put for a short signal,
3. choose the option with absolute delta closest to 0.70,
4. size contracts from fixed risk capital,
5. apply hard stop and take-profit rules.

This rule is not a neural model. It is promoted because it is robust and because learned alternatives did not beat it in the current tests.

9.2 Why Fixed Delta Can Beat Learned Selection

Option strike selection is path-dependent and extremely noisy. A learned selector must infer premium response, spread sensitivity, gamma convexity, volatility change, and liquidation timing. The fixed $\Delta = 0.70$ rule is less flexible, but it avoids many degrees of freedom and therefore currently generalizes better.

10 OptionValueJEPa Research Architecture

OptionValueJEPa is a separate neural model for option candidate valuation. It receives:

- a market feature block, including `base_jepa` signal features,
- an option candidate block, including delta, premium, spread, strike distance, IV, theta, and gamma,
- a dynamic state block during an open trade, including current PnL, premium ratio, MFE/MAE-like fields, delta drift, IV, gamma, spot return, and minutes remaining.

The architecture has three encoders:

$$z_m = E_m(x_m), \quad z_o = E_o(x_o), \quad z_d = E_d(x_d).$$

The entry path predicts three candidate values:

$$\left[\hat{V}_{\text{hold180}}, \hat{V}_{\text{rule}}, \hat{V}_{\text{best}} \right] = H_{\text{entry}}([z_m, z_o, T(z_m, z_o, h)]).$$

The dynamic path predicts continuation value:

$$\hat{V}_{\text{continue}} = H_{\text{dyn}}([z_m, z_o, z_d, h_{\text{remaining}}]).$$

It is trained with SmoothL1 loss on clipped return-on-risk targets. This architecture is conceptually correct, but the current experiments show it has not yet closed the gap to the oracle.

11 Oracle Definitions

The reports include oracle rows. These are not models. They use future information and serve only as upper bounds:

- **oracle best delta hard:** chooses the best delta bucket after observing the future path, then exits with mechanical rules.
- **oracle best delta oracle exit:** chooses both the best candidate and best future exit point.

Large oracle performance does not imply deployable edge. It only indicates potential head-room if a learnable signal can approximate the oracle without using future information.

12 Live Runtime

The live runtime has two primary scripts:

- `services/realtime_feed.py` generates live features compatible with the production `base_jepa` model.
- `bots/tradingbot_wrapper_jepa.py` loads the production direction model and applies the fixed-delta option execution rule.

Model artifacts are under `neural/models/jepa`. They are ignored by git and deployed with `push_models.ps1`. Source code and systemd service files are deployed through git.

13 Empirical Summary

The architectural conclusion is specific: JEPA adds value as an auxiliary representation to the GBT. JEPA-only and learned option selection are not currently sufficient replacements.

Model	Trades	WR	PF	Avg bps	PnL	Max DD
base	80	58.8%	1.389	4.40	\$3,522	-\$1,640
jepa_only	110	59.1%	1.333	3.86	\$4,245	-\$3,270
base_jepa	105	68.6%	1.945	8.70	\$9,132	-\$1,522

Table 3: Strict frozen-March April–May 2026 OOS 180 minute proxy backtest.

14 Risk Controls and Bias Controls

The architecture contains explicit controls:

- future return columns are excluded from feature selection,
- JEPA context validity is required before a row can be scored,
- strict OOS tests freeze training before April 2026,
- production diagnostics are labelled as non-OOS,
- oracle rows are labelled as non-deployable,
- live artifacts store medians, features, thresholds, and metadata.

15 Conclusion

The current architecture is a supervised market-state predictor plus deterministic option execution, not an RL system. The JEPA layer is valuable because it converts recent state/input dynamics into latent features that improve a tabular GBT. The fixed-delta option execution rule remains the production choice because the learned option-value and exit models have not yet shown robust superiority. Future work should focus on improving OptionValueJEPA with better path labels, ticker-specific calibration, slippage modelling, and walk-forward promotion gates rather than replacing the already-promoted `base_jepa` direction model prematurely.